

PORTADA

FALTA MODO AUTOMATICOY MANUAL COMO PASAR DE UNO A OTRO ... Y NUEVO
METER WALL

Índice

Índice.....	2
1.Introducción a SURI.....	3
1.1 ¿Qué es la Plataforma SURI?.....	3
1.2 Enfoques de uso: Usuario y Desarrollador.....	3
1.3 Modos de simulación: Virtual y Físico.....	3
1.4 Tecnologías empleadas y su justificación.....	4
Arquitectura del Sistema.....	4
¿Por qué estas tecnologías frente a otras alternativas?.....	4
2. Primeros Pasos y Configuración.....	6
2.1 Requisitos del sistema.....	6
2.2 Instalación y compilación del entorno.....	6
2.3 Ejecución del código base.....	6
2.4 Entendiendo la interfaz gráfica:.....	7
2.5 Interfaz HMI y control del robot físico.....	7
3. Metodología de Uso.....	9
3.1 Modo Comandos (Ejecución vía terminal).....	9
3.2 Modo Rutina (Ejecución de archivos .yaml).....	9
4. Pruebas y Casos de Uso Prácticos.....	10
4.1 Movimientos cinemáticos básicos.....	10
4.2 Posición segura: HOME.....	10
4.3 Mover articulaciones por separado.....	11
4.4 Gestión de Herramientas: Añadir y probar un nuevo TCP.....	11
4.5 Gestión de Colisiones: Creación de paredes virtuales.....	11
4.6 Caso de Estudio Avanzado: Jugar al 3 en Raya.....	12
○ 4.6.1 Configuración del entorno y Machine Learning (PyTorch).....	12
○ 4.6.2 Configuración de la visión (Cámaras y Tablero).....	12
○ 4.6.3 Ejecución de la partida e interacción.....	13
5. Referencia de Funcionalidades (Comandos y Servicios ROS2).....	14
● 5.1 Gestión del TCP (Herramientas):.....	14
● 5.2 Gestión del Entorno (Obstáculos):.....	14
● 5.3 Gestión de Visión y Objetos:.....	14
● 5.4 Ejecución de Rutinas:.....	14
6. Guía Básica para Desarrolladores.....	15
6.1 Cómo cambiar el modelo del robot.....	15
6.2 Comunicación serial con el robot físico.....	15
6.3 Añadir funcionalidades y Módulos Adicionales (DLCs).....	16
7.Anexos.....	16

1.Introducción a SURI

1.1 ¿Qué es la Plataforma SURI?

SURI (**S**imulación **U**niversal para **R**obótica **I**ndustrial) es una plataforma de simulación, control y aprendizaje para brazos robóticos. Tiene una interfaz amigable para el usuario y una escalabilidad grande que permite a los estudiantes seguir formándose y crear nuevas funcionalidades. Lo que diferencia a esta plataforma de otras es la facilidad de cambiar, añadir o eliminar configuraciones. Tiene un diseño modular que permite cambiar, añadir o eliminar partes muy fácilmente sin afectar al resto del programa.

Una de las características más destacadas es la facilidad de cambiar el robot. No se necesita una configuración o conocimientos muy avanzados. Tan solo el urdf del robot, que se puede encontrar en internet o crear uno propio con mucha facilidad.

1.2 Enfoques de uso: Usuario y Desarrollador

Este proyecto tiene dos enfoques de uso:

Usuario: Proporciona una interfaz de alto nivel enfocada en la usabilidad. Permite enviar comandos intuitivos (movimiento de coordenadas, control de pinzas) y visualizar la ejecución en tiempo real sin necesidad de un alto conocimiento en robótica. Enfocado al uso de las funcionalidades ya preprogramadas.

Desarrollador: Ofrece una base de código modular diseñada para ser ampliada. Permite desde programar e integrar nuevas funcionalidades a medida, hasta cambiar el modelo del robot por completo de forma ágil y sin reestructurar el sistema, adaptándose a las necesidades de cualquier proyecto.

1.3 Modos de simulación: Virtual y Físico

Este proyecto tiene varias maneras de ejecutarse

Solo virtual: Todo será virtual mediante Pybullet que simulará todos los movimiento del robot, con la escena (objetos, cámaras...) necesaria. Perfecto para hacer pruebas y validación

Solo real: Permite controlar el robot sólo físicamente mediante arduino, en el ordenador solo se verán mensajes de depuración o información, no se verá ninguna simulación. En caso de poner en producción y no se necesita una simulación porque ya es un programa automatizado y controlado. También es útil si quieres manejar más físico y mecánica que software.

Híbrido: Funciona tanto real como virtual, haciendo que la simulación funcione como un gemelo digital a tiempo real. Perfecto para comprobar que funciona bien todo, permite visualización para depurar y manejo real del robot.

Gracias a su estructura modular permite que se pueda funcionar de varias maneras y añadir funcionalidades sin problema.

1.4 Tecnologías empleadas y su justificación

Para lograr que SURI sea accesible, modular y no requiera un superordenador, se ha seleccionado cuidadosamente las tecnologías base. Cada pieza cumple una función específica y ha sido elegida pensando en mantener una curva de aprendizaje amigable y fluida.

Arquitectura del Sistema

- **Motor de Físicas y Simulación (PyBullet):** Juega un papel doble en SURI. Por un lado, trabajando en su modo interno (DIRECT), funciona como el cerebro matemático: se encarga de calcular la cinemática, evitar las singularidades y detectar colisiones. Por otro lado, en su modo gráfico (GUI), es el encargado de dar vida al gemelo digital: renderiza el modelo visual del robot, las cámaras y los objetos del entorno para que el usuario pueda ver la simulación en tiempo real.
- **Control y Comunicaciones (ROS 2 Jazzy):** Es el sistema nervioso de la plataforma y el estándar actual en la industria robótica. Se encarga de comunicar todos los módulos entre sí. Aquí es donde se gestionan y guardan herramientas (como el TCP), se añaden nuevos paquetes y se ejecutan las rutinas o casos de uso más avanzados (como jugar al 3 en raya).
- **Motor de Hardware (Arduino):** Es el encargado de controlar el brazo robótico físico. Actúa como puente: se comunica con ROS 2 a través del puerto USB para informar de la posición actual del robot y recibir las coordenadas hacia las que debe moverse.

¿Por qué estas tecnologías frente a otras alternativas?

La premisa principal de SURI es la facilidad de uso, concretamente, poder cambiar de robot simplemente sustituyendo un archivo .urdf sin necesidad de configuraciones avanzadas. Basándonos en esto, se han evaluado y descartado otras opciones populares en el sector:

Alternativa Evaluada	Motivo del descarte en el contexto de SURI
Movelt / MuJoCo	Tienen una curva de aprendizaje muy empinada. Sus complejas configuraciones opacan el objetivo de sencillez y rapidez de SURI.

Pinocchio FCL	+	Exigen un nivel de complejidad matemática y de programación que no encaja con la interfaz amigable que buscamos.
Isaac Sim		Muy específico y excesivamente pesado; requiere recursos de hardware (como GPUs de alta gama) innecesarios para este proyecto.
Gazebo		Consume bastantes recursos para ordenadores estándar y la relación entre su peso y sus visuales no compensa frente a alternativas más ligeras.
CoppeliaSim		No siempre garantiza ser de código abierto y tiene una curva de aprendizaje media.
Webots		Aunque es una alternativa excelente y fácil de implementar, PyBullet demostró ser el punto de equilibrio perfecto. Es más ligero para entornos con limitaciones de hardware y permite importar URDFs de forma mucho más directa y veloz.

Por su parte, la elección de **Arduino** para el control físico está justificada por su bajo coste, su altísima escalabilidad y su facilidad de uso, lo que permite una personalización casi total sin encarecer el proyecto.

De esta manera, se obtiene una plataforma con una curva de aprendizaje baja, que no requiere de grandes inversiones económicas ni ordenadores de última generación, siendo a la vez muy flexible, modular y fácilmente personalizable.

2. Primeros Pasos y Configuración

Esta sección es una guía del proceso de instalación, compilación y puesta en marcha de la plataforma. El objetivo es dejar el entorno completamente operativo en el equipo local.

2.1 Requisitos del sistema

Para que SURI funcione correctamente, es necesario tener instalados los siguientes componentes en la máquina local. En caso de usar Windows, es posible ejecutarlo utilizando el subsistema de Linux (WSL2):

- **Sistema Operativo:** Ubuntu 24.04 (Nativo o en WSL2).
- **ROS 2 Jazzy:** El núcleo de comunicaciones de la plataforma.
- **Pybullet:** Para el entorno del simulador de físicas.
- **RViz2:** (Normalmente se instala automáticamente con la versión *Desktop* de ROS 2).
- **Arduino IDE:** Necesario para compilar y subir el código al controlador del robot físico.

2.2 Instalación y compilación del entorno

Para empezar, se requiere descargar el código fuente y compilar el espacio de trabajo de ROS 2. Hay que abrir una terminal y ejecutar las siguientes líneas una a una:

1. Clonar el repositorio

```
git clone https://github.com/SilCalvo/TFG.git
```

2. Entrar en la carpeta del espacio de trabajo (workspace)

```
cd TFG/src/ros2_ws
```

3. Compilar los paquetes de ROS 2

```
colcon build
```

4. Cargar el entorno para que la terminal reconozca los comandos

```
source install/setup.bash
```

Nota: El paso 4 (`source install/setup.bash`) debe ejecutarse cada vez que se abra una terminal nueva, a menos que se añada al archivo `.bashrc` del sistema.

2.3 Ejecución del código base

Existen diferentes formas de lanzar el código de SURI dependiendo de las necesidades. Se debe elegir el modo de ejecución que mejor se adapte al objetivo en cada momento.

Solo Físico: Ideal cuando el robot ya está en producción y no se necesita simulador.

comando:

Solo Virtual: Perfecto para hacer pruebas y validar trayectorias sin riesgo de colisiones reales.

comando:

Híbrido (Gemelo Digital): Conecta el robot físico con la simulación en tiempo real.

comando:

2.4 Entendiendo la interfaz gráfica:

Al lanzar la simulación, se trabajará con dos ventanas principales que tienen propósitos muy distintos:

- **2.4.1 RViz:** Visualización de ejes, TFs y modelo del robot

Aquí no se muestra un entorno fotorrealista. RViz visualiza la "intención" del robot. Es la herramienta ideal para ver los ejes de coordenadas (Frames/TFs), planificar trayectorias, observar la estructura cinemática y depurar el funcionamiento interno de ROS 2

- **2.4.2 Pybullet:** El gemelo digital

Este se ve un entorno que se puede observar las pareces virtuales, meter objetos en escena, camaras rgb... se mueve al recibir los comandos que los recibe a la vez que el robot real.

2.5 Interfaz HMI y control del robot físico

La plataforma SURI permite el control de un brazo robótico físico mediante un panel de control o interfaz HMI (*Human-Machine Interface*) gestionado por una placa Arduino.

Montaje y configuración inicial: Para el ensamblaje del robot real, se deben seguir las instrucciones detalladas en el [vídeo de montaje - añadir enlace aquí]. La única modificación respecto al diseño del vídeo original es la sustitución del motor paso a paso por un servomotor MSG90. Es importante recordar que las dimensiones de esta modificación deben actualizarse en el archivo .urdf. El esquema eléctrico completo para realizar las conexiones se encuentra disponible en el apartado de Anexos.

Controles físicos: El panel está diseñado para facilitar el manejo manual y la interacción con la simulación. Sus componentes principales son:

- **Potenciómetros:** Permiten mover cada articulación de forma independiente. El sistema interpola los datos de lectura de cada potenciómetro para traducirlos exactamente a los grados de movimiento de los servomotores.

- **Botonera (3 botones):**

- Botón para activar el **Modo Automático** (control del robot desde el ordenador).
- Botón para activar el **Modo Manual** (control físico mediante los potenciómetros).
- Botón de **Envío de puntos (pared virtual)**: Al operar en modo manual, pulsar este botón envía las coordenadas actuales del robot al sistema.

Indicadores visuales (LEDs): El panel dispone de un sistema de iluminación para identificar rápidamente el estado operativo del robot:

- **LED Azul:** Se mantiene encendido cuando el Modo Automático está activo.
- **LED Amarillo:** Se mantiene encendido cuando el Modo Manual está activo.
- **LED Verde:** Emite un parpadeo (se enciende y se apaga) como confirmación visual cada vez que se guarda un punto con el botón de envío.

Sistema de suavizado de transición: Una característica clave de la interfaz física es su mecanismo de seguridad al cambiar de Modo Automático a Modo Manual.

Durante el funcionamiento automático, el robot puede moverse a posiciones muy distintas a las que marcan los potenciómetros en ese momento. Si se cambiara al modo manual de forma directa, el robot intentaría corregir su posición de golpe para coincidir con los mandos, provocando un salto brusco. Para evitarlo, el sistema realiza una transición suave: mueve cada articulación gradualmente desde su posición actual hasta la posición que indican los potenciómetros.

Mientras este ajuste gradual se está llevando a cabo, **los LEDs azul y amarillo permanecerán encendidos simultáneamente** hasta que el robot finalice el movimiento y pase de forma definitiva al control manual.

3. Metodología de Uso

La plataforma SURI ofrece dos metodologías principales para enviar instrucciones al robot. La elección entre una u otra dependerá de si se busca realizar pruebas puntuales o automatizar un proceso completo.

3.1 Modo Comandos (Ejecución vía terminal)

Este modo permite una interacción directa y manual con el sistema a través de la terminal de comandos.

Consiste en introducir las instrucciones de movimiento o acción una a una. Al ser un sistema de ejecución secuencial, el terminal queda a la espera y no permite procesar un nuevo comando hasta que el robot haya finalizado por completo la acción anterior. (La lista completa y la sintaxis de estos comandos se detallan más adelante, en la Sección 5).

¿Para qué se recomienda? Es el método ideal para la validación y el diagnóstico. Resulta muy útil para probar coordenadas específicas, verificar el alcance del brazo robótico o confirmar que una acción hace exactamente lo que se espera antes de integrarla en un proceso más largo.

3.2 Modo Rutina (Ejecución de archivos .yaml)

Este modo está diseñado para la automatización de tareas. En lugar de escribir los comandos uno a uno, el sistema lee y ejecuta de forma secuencial una lista de instrucciones predefinidas guardadas en un archivo de configuración con formato .yaml.

Para crear y ejecutar una rutina, los archivos .yaml deben alojarse en el directorio correspondiente del proyecto:

```
robot_ws/src/robot_pkg/rutines
```

Recomendación de seguridad: Antes de ejecutar una rutina completa por primera vez, es altamente recomendable probar los comandos y las posiciones clave por separado utilizando el **Modo Comandos**. Esto garantiza que las trayectorias son seguras, físicamente posibles para el robot y que no existen riesgos de colisión en el entorno.

4. Pruebas y Casos de Uso Prácticos

Una vez configurado el entorno, es el momento de poner a prueba el sistema. En esta sección se detallan las operaciones fundamentales que puede realizar el brazo robótico, desde movimientos básicos hasta la integración de inteligencia artificial.

4.1 Movimientos cinemáticos básicos

El brazo robótico puede planificar sus trayectorias de diferentes maneras. Es fundamental entender la diferencia entre los dos tipos de movimiento principales para evitar errores durante la ejecución:

MoveJ (Movimiento Articular o *Joint*): El robot calcula la forma más rápida de llegar desde la posición actual hasta el punto final, sin importar el camino que dibuje en el aire.

- Característica: La orientación del Punto Final de la Herramienta (TCP) va cambiando a lo largo del trayecto.
- Cuándo usarlo: Es ideal para movimientos rápidos cuando el robot va vacío. Si físicamente es posible llegar al destino, este comando casi siempre tendrá éxito.
- Cuándo NO usarlo: No se debe usar si el robot transporta algo que debe mantenerse siempre nivelado (por ejemplo, un vaso lleno de líquido o una caja abierta), ya que al girar durante el trayecto, el contenido se caería.

MoveL (Movimiento Lineal): El robot se mueve trazando una línea recta perfecta entre el punto de inicio y el destino, manteniendo una orientación fija del TCP en todo momento.

- Característica: Es un movimiento muy estricto y matemáticamente complejo.
- El problema: Puede ocurrir que, aunque el robot sea físicamente capaz de alcanzar el punto final con la orientación deseada, no pueda trazar la línea recta intermedia debido a las limitaciones de sus articulaciones (lo que en robótica se conoce como "singularidad cinemática"). En estos casos, el movimiento fallará y se cancelará por seguridad.

4.2 Posición segura: HOME

Uno de los pasos más importantes en cualquier rutina es establecer una posición de "Home" o reposo seguro (generalmente con el brazo robótico completamente extendido hacia arriba).

Esta posición no solo sirve para guardar el robot, sino que actúa como un "reseteo" mecánico. Muchos servomotores pueden girar más de 360 grados; aunque visualmente parezca que el robot está en una postura normal, internamente el motor podría estar al borde de su límite de giro físico o a punto de tensar los cables. Enviar el robot a la posición

HOME asegura que todos los contadores de las articulaciones se reinicien, garantizando que el siguiente movimiento parta de un estado seguro y conocido.

4.3 Mover articulaciones por separado

Si se requiere un control más fino o depurar un motor específico, es posible mover cada articulación de forma individual.

Para ello, basta con seleccionar el índice de la articulación (la numeración empieza en 0 para la base y va subiendo hasta la punta) e introducir los grados a los que se desea que gire (generalmente en un rango de 0 a 180 grados, dependiendo del motor).

4.4 Gestión de Herramientas: Añadir y probar un nuevo TCP

El **TCP** (*Tool Center Point* o Punto Central de la Herramienta) es el punto exacto de la herramienta que el robot utiliza para interactuar con el entorno. Para el sistema de control, este punto virtual es el verdadero final del brazo robótico, y todos los cálculos matemáticos de movimiento y orientación se realizan tomando exclusivamente este punto de referencia.

La plataforma SURI es altamente modular y permite añadir, cambiar o eliminar herramientas en plena ejecución. Sin embargo, al cambiar de herramienta cambian sus dimensiones. Para que el robot se mueva con precisión geométrica y no choque, se debe actualizar la configuración para reubicar el TCP en el nuevo extremo exacto de interacción:

- Si se usa un rotulador, el TCP debe definirse exactamente en la punta que tocará el papel.
- Si se usa una pinza (*gripper*), el TCP debe establecerse en el espacio central exacto donde los dedos agarran el objeto.

4.5 Gestión de Colisiones: Creación de paredes virtuales

La seguridad es el factor más crítico en robótica. Tradicionalmente, esto se soluciona instalando una celda física de metal alrededor del robot; si un humano entra, la máquina se apaga de golpe. Sin embargo, esto requiere construcción física, ocupa espacio y anula la interactividad.

SURI utiliza un enfoque mucho más moderno: las **paredes virtuales** (geofencing). El sistema permite crear barreras invisibles en la escena 3D que el robot tiene prohibido atravesar bajo cualquier circunstancia. La gran ventaja es su flexibilidad: las paredes se

pueden añadir, modificar o eliminar en tiempo real durante la ejecución. Esto garantiza que un usuario siempre estará seguro al otro lado de la barrera virtual, sin necesidad de instalaciones físicas costosas y permitiendo que el entorno de trabajo cambie dinámicamente.

4.6 Caso de Estudio Avanzado: Jugar al 3 en Raya

Para demostrar la capacidad de SURI al integrar robótica, visión artificial y toma de decisiones, se ha incluido un módulo adicional (DLC) que permite jugar partidas de "3 en raya" (Tic-Tac-Toe) contra el robot.

La ejecución de este caso requiere configurar varios elementos del entorno:

- 4.6.1 Configuración del entorno y Machine Learning (PyTorch)

Para que el robot sea capaz de distinguir entre una "X" y una "O", se utiliza un modelo ligero de Inteligencia Artificial (Machine Learning) que clasifica las piezas visualmente. Para evitar problemas de compatibilidad y facilitar su uso, se ha preparado un entorno virtual de Python que contiene todas las librerías necesarias (como PyTorch).

Para activar el entorno y cargar el modelo de clasificación, ejecuta:

Añadir comandos de ejecución

- 4.6.2 Configuración de la visión (Cámaras y Tablero)

El siguiente paso es añadir la escena de juego. Esto despliega el tablero virtual y posiciona una cámara RGB-D. El uso de este tipo de cámara es clave: captura los colores para identificar las piezas dibujadas o colocadas, y utiliza la profundidad (Depth) para calcular la distancia exacta a la que se encuentran en el espacio 3D.

Para cargar el tablero y la cámara en la simulación, ejecuta:

añadir comando de ejecucion

○ 4.6.3 Ejecución de la partida e interacción

Una vez que el entorno físico, la visión y la inteligencia artificial están funcionando, se puede iniciar el bucle del juego. El sistema analizará el estado del tablero mediante la cámara, decidirá su próximo movimiento y el brazo robótico colocará su pieza.

Para interactuar y colocar tus piezas, debes seguir los siguientes pasos:

añadir comando de ejecucion

5. Referencia de Funcionalidades (Comandos y Servicios ROS2)

- **5.1 Gestión del TCP (Herramientas):**

- Añadir TCP (`/add tool`)
- Seleccionar TCP Activo (`active_tool`)
- Borrar TCP

- **5.2 Gestión del Entorno (Obstáculos):**

- Añadir pared (`/add_wall`)
- Borrar pared (`/remove_wall`)

- **5.3 Gestión de Visión y Objetos:**

- Añadir cámara (`/add_camera`)
- Borrar cámara
- Añadir objeto (`/add_object`)
- Borrar objeto

- **5.4 Ejecución de Rutinas:**

- Lanzar nodo de rutina (`routine_node`)

6. Guía Básica para Desarrolladores

Esta sección está orientada a usuarios con un perfil técnico o desarrolladores que deseen escalar la plataforma. Aunque poseer conocimientos previos de ROS 2 facilita enormemente el proceso, la arquitectura modular de SURI está diseñada para que implementar modificaciones estructurales sea lo más intuitivo posible, evitando reescribir código complejo.

6.1 Cómo cambiar el modelo del robot

Una de las grandes ventajas de SURI es que, para cambiar el modelo cinemático y visual del robot, no es necesario alterar el motor matemático. El proceso consta de dos únicos pasos:

1. **Importar el modelo 3D:** Se debe colocar el archivo `.urdf` del nuevo brazo robótico en el directorio correspondiente y renombrarlo obligatoriamente como `robot.urdf`:
`src/ros2_ws/src/robot_pkg/urdf/robot.urdf`

2. **Actualizar los parámetros:** Es necesario modificar el archivo de configuración para indicarle al sistema las características del nuevo brazo:
`src/ros2_ws/src/robot_pkg/config/robot_params.yaml`

6.2 Comunicación serial con el robot físico

SURI utiliza Arduino por defecto para el control de los motores, pero su arquitectura es agnóstica respecto al hardware. Esto permite sustituir el microcontrolador por cualquier otro (como un ESP32, un STM32, etc.).

Para realizar este cambio, la única pieza de software que necesita ser adaptada es el nodo escrito en C++ que actúa como puente de comunicación serial entre ROS 2 y la placa física. Este archivo se encuentra en:

`src/ros2_ws/src/robot_pkg/src/servo_bridge_node.cpp`

Este nodo se encarga de abrir el puerto serie y establecer la velocidad de transmisión de datos. Por defecto, está configurado para operar a una velocidad de **57600 baudios** (B57600), coincidiendo con la configuración del código de Arduino. Si el nuevo microcontrolador utiliza una tasa de baudios diferente o se conecta en un puerto distinto, bastará con localizar y ajustar estos dos parámetros en el código y volver a compilar el entorno.

6.3 Añadir funcionalidades y Módulos Adicionales (DLCs)

La escalabilidad del proyecto permite dos vías principales de desarrollo. Es importante diferenciar entre ampliar las capacidades del sistema y crear aplicaciones de usuario final:

- **Nuevas Funcionalidades (Core):** Son herramientas u opciones que mejoran el control interno del robot, pero no constituyen un proyecto o tarea final. Algunos ejemplos serían: programar un nuevo tipo de movimiento (como MoveC para trayectorias circulares), mejorar los algoritmos de evasión de colisiones o añadir un sistema para recalcular el TCP de forma dinámica desde el modo manual.
- **Módulos Adicionales (DLCs o Casos de Uso):** Consiste en utilizar la plataforma SURI y todas sus herramientas para desarrollar un proyecto completo y automatizado. El juego del "3 en raya" incluido en la plataforma es un ejemplo de DLC.

El objetivo a largo plazo es que la comunidad pueda crear y añadir nuevos paquetes (DLCs) que hagan uso de la base de SURI. Para asegurar una correcta integración de nuevos nodos, se recomienda consultar el **esquema de la arquitectura de ROS 2** disponible en los Anexos.

Ideas de proyectos (DLCs) para practicar: Para familiarizarse con el desarrollo en la plataforma, algunas propuestas interesantes de escalabilidad podrían ser:

- **Robot *Bartender*:** Una rutina que reciba comandas, localice vasos y mezcle líquidos mediante posiciones predefinidas.
- **Robot "Picasso":** Un sistema que procese una fotografía mediante visión artificial y genere las trayectorias necesarias para que el robot dibuje con un rotulador.
- **Clasificador industrial:** Uso de cámaras e IA para identificar piezas de distintos colores o defectuosas en una zona de trabajo y separarlas en diferentes cajas.

7. Anexos